

What are Storage Classes?

In C, a **storage class** defines four key properties for any variable:

Storage Location: Where the variable's memory is reserved (Stack, CPU Registers, or Data Segment).

Default Initial Value: The default value initialized by the system if you don't assign one explicitly (e.g., 0 vs. garbage values).

Scope (Visibility): The region of the code where the variable can be directly accessed (Block, Function, or Program scope).

Lifetime (Longevity): How long the variable retains its allocated memory space during program execution.

Variable Scope Levels

Block Scope: Accessible only within the { ... } block where declared.

Function Scope: Accessible throughout the enclosing function.

Program (Global) Scope: Accessible across all functions and files within the program.

Summary Comparison Matrix

Storage Class	Keyword	Default Value	Storage Location	Scope	Lifetime
Automatic	auto	Garbage Value	Stack Segment	Block / Local	Until end of current block
Register	register	Garbage Value	CPU Register	Block / Local	Until end of current block
Static	static	Zero (0)	Data Segment	Block / Local	Until program

Storage Class	Keyword	Default Value	Storage Location	Scope	Lifetime
					terminates
External	extern	Zero (0)	Data Segment	Program / Global	Until program terminates

The 4 Storage Classes in Detail :

1. Automatic Storage Class (auto)

Behavior: The default storage class for local variables declared inside any function or block. Writing `auto int x;` is identical to writing `int x;`.

Memory Placement: Allocated on the **Stack Segment**.

```
#include <stdio.h>
int main()
{
    auto int x = 10; -----> Explicit auto declaration
    {
        auto int y = 20; -----> Scope is restricted to this inner block
    }
    -----> y is destroyed here (out of scope)
    return 0;
}
```

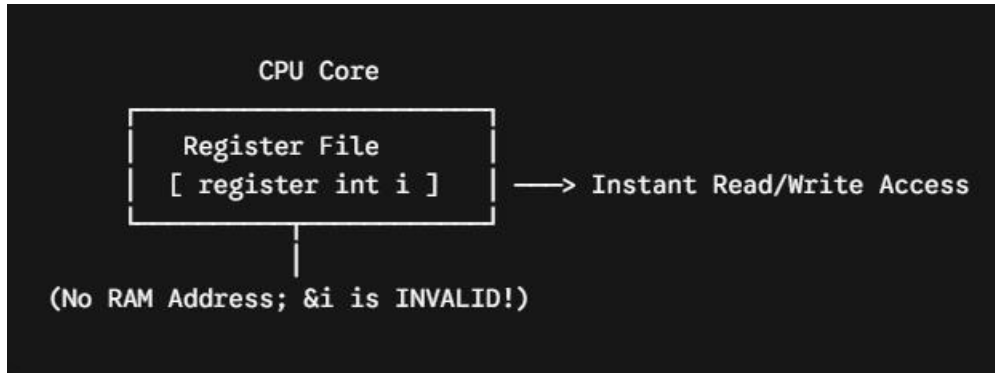
2. Register Storage Class (register)

Behavior: Hints to the compiler to store the variable directly inside a **CPU Register** instead of RAM (Stack).

Purpose: Dramatically cuts access times for frequently referenced variables (such as loop counters).

Fallback Rule: If no free CPU registers are available, the compiler automatically falls back to treating the variable as an auto local variable.

Critical Limitation: You **cannot** use the address-of operator (&) on a register variable. CPU registers do not have memory addresses in RAM.



```
int main()
{
    register int i; -----> Request CPU register space
    int *ptr = &i; -----> COMPILER ERROR! Cannot get address of register variable
}
```

3. Static Storage Class (static)

Behavior: Preserves its stored value across multiple function calls.

Key Traits:

Initialized **only once** when execution reaches the variable definition. Subsequent call cycles skip re-initialization.

Allocated in the **Data Segment**, keeping it active until the entire program terminates.

Scoped strictly within its local block or function.

Call 1 —> Initialized once: x = 0 —> Modified: x += 10 —> Output: x = 10

Call 2 —> Reuses stored value: x = 10 —> Modified: x += 10 —> Output: x = 20

```
#include <stdio.h>
```

```

void display()
{
    static int x = 0; ----> Initialized ONLY ONCE at program startup
    x += 10; -----> Value persists between function calls
    printf("x = %d\n", x);
}
int main()
{
    display(); -----> Outputs: x = 10
    display(); -----> Outputs: x = 20
    return 0;
}

```

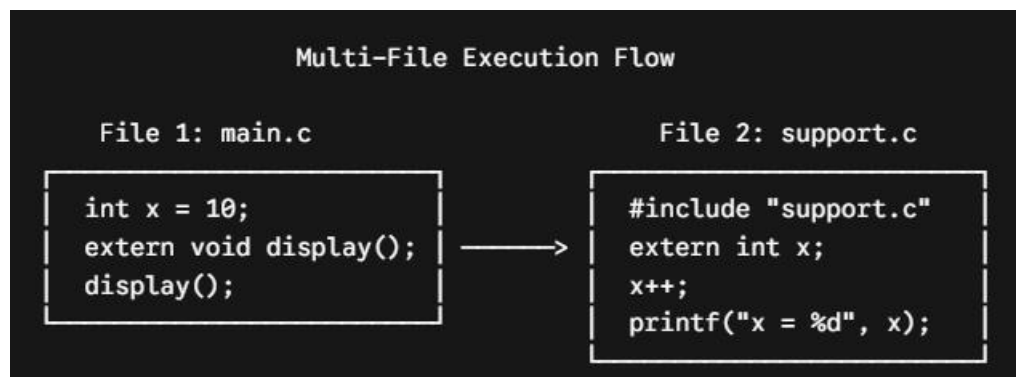
4. External Storage Class (extern)

Behavior: Declares a global variable that is defined elsewhere (in another file or later in the same file).

Declaration vs. Definition:

`int var;` — **Definition** (allocates memory space).

`extern int var;` — **Declaration** (tells the compiler the variable exists elsewhere without allocating new memory).



Code Implementation across files:

support.c

```

#include <stdio.h>
extern int x; -----> References global variable x defined in main file
void display()
{

```

```
x++; -----> Modifies shared global x  
printf("Hello from support file. x = %d\n", x);  
}
```

main.c

```
#include <stdio.h>  
#include "support.c" -----> Including secondary implementation file  
int x = 10; -----> Definition of global variable x  
extern void display(); -----> Explicit function declaration  
int main()  
{  
    display(); -----> Calls function from support.c; outputs: x = 11  
    return 0;  
}
```

Note:

For making your understanding more clear and simple, you must visit the handwritten note provided in the same folder **16-Storage_Classes**.